



Fleet Management System

Rapports d'Audit Technique

DATE	11/06/2026
VERSION	2.0
BRANCHE	main
PRÉPARÉ PAR	Claude Code — Anthropic

01 - Architecture & Patterns

02 - Sécurité

03 - Performance, Modernité & Scalabilité

04 - Tests

Table des matières

Rapport 01 — Architecture	3
1.1 Vue d'ensemble & Stack technique	3
1.2 Architecture Frontend	4
1.3 Architecture Backend	5
1.4 Infrastructure & Déploiement	6
Rapport 02 — Sécurité	7
2.1 Tableau de bord sécurité	7
2.2 Problèmes critiques	8
2.3 Points forts	9
2.4 Recommandations	10
Rapport 03 — Performance, Modernité & Scalabilité	11
3.1 Performance Frontend & Backend	11
3.2 Modernité de la stack	12
3.3 Scalabilité	13
3.4 Synthèse globale	14
Rapport 04 — Tests	15
4.1 Tests Backend — PHPUnit	15
4.2 Tests Frontend — Vitest	16
4.3 Synthèse de la couverture	17

1.1 Vue d'ensemble

Fleet Management System est une application web de gestion de flotte de véhicules organisée en **monorepo** avec une séparation stricte frontend/backend. L'architecture suit le paradigme **SPA + REST API**.



Stack technique complète

Couche	Technologie	Version	Rôle
Frontend	React	19.1.0	Framework UI
Build	Vite	6.3.5	Bundler + dev server
Routing	React Router DOM	7.6.2	Navigation SPA
UI	Bootstrap + React Bootstrap	5.3.8 / 2.10.10	Composants UI
HTTP	Axios	1.10.0	Client HTTP + intercepteurs
i18n	react-i18next	15.6.1	Internationalisation EN/FR
Animations	Framer Motion	12.23.24	Transitions UI
Exports	jsPDF + XLSX	4.2.1 / 0.18.5	PDF & Excel côté client
Backend	Laravel	12.0	Framework API MVC
Auth API	Laravel Sanctum	4.1	Tokens + sessions
Langage	PHP	8.2+	Runtime backend
Base de données	Neon (PostgreSQL)	16.x	Persistance serverless
CI/CD	GitHub Actions	—	Lint, tests, déploiement

1.2 Architecture Frontend

Pattern : SPA (Single Page Application)

Le frontend est une SPA React avec routage côté client. L'état global se limite au contexte d'authentification via **React Context API** — pas de Redux ni Zustand, ce qui est adapté à la taille actuelle de l'application.

```
fleet-mgt-frontend/src/
├─ context/      # État global (AuthContext)
├─ components/   # 49 composants JSX réutilisables
│  └─ Layouts/   # MainLayout, Sidebar, Header, Footer
│  └─ Auth/      # PrivateRoute, LogoutButton
│  └─ UI/        # Modal, Alert, Loader, FormInput, Pagination
├─ pages/        # Vues liées aux routes
├─ hooks/        # Custom React hooks
├─ utils/        # Fonctions utilitaires
├─ locales/      # Traductions EN / FR
└─ axios.js      # Client HTTP centralisé
```

Arborescence des routes

Route	Accès	Rôles autorisés
/login, /register, /forgot-password	Public	Tous
/dashboard, /profile	Protégé	Tous les rôles
/vehicles/*	Protégé	admin, manager, accountant
/maintenances/*	Protégé	admin, manager
/consumptions/*	Protégé	admin, manager, accountant
/reports/*	Protégé	admin, manager, accountant
/users/*	Protégé	admin uniquement

Flux d'authentification



Timer d'inactivité : avertissement à 28 minutes, déconnexion automatique à 30 minutes (SessionWarningModal).

1.3 Architecture Backend

Pattern : MVC (Model-View-Controller) — Laravel

```
fleet-mgt-api/app/
├─ Http/
│  └─ Controllers/      # 8 contrôleurs (Auth, User, Vehicle, Maintenance...)
│     └─ Middleware/    # SecurityHeaders, CORS, Authentification
├─ Models/              # 5 modèles Eloquent ORM
├─ Providers/           # AuthServiceProvider (RBAC Gates)
└─ config/              # Sanctum, CORS, Auth, Cache...
```

Modèle de données

Table	Champs clés	Relations
users	id, name, email, role, login_attempts, blocked_until	hasMany: vehicles, reports
vehicles	id, license_plate, marque, model, fuel_type, status, current_driver_id	belongsTo: user; hasMany: maintenances, consumptions
maintenances	id, vehicle_id, type, date, cost, notes	belongsTo: vehicle
consumptions	id, vehicle_id, fuel_amount, date, cost	belongsTo: vehicle
reports	id, vehicle_id, report_type, title, metadata (JSON)	belongsTo: vehicle

RBAC — Rôles et permissions

Géré via **Laravel Gates** dans `AuthServiceProvider` :

Rôle	Périmètre d'accès
admin	Accès total au système
manager	Véhicules, maintenances, rapports, utilisateurs (lecture)
accountant	Véhicules (lecture), consommations, rapports
driver	Véhicules assignés, consommations propres
mechanic	Maintenances uniquement

1.4 Infrastructure & Déploiement

Environnements

Couche	Développement	Production
Frontend	Vite dev server (port 5173)	Cloudflare Pages (CDN global)
Backend	PHP Artisan (port 8000)	Railway (Docker auto-build)
Base de données	PostgreSQL Docker (port 5432)	Neon (PostgreSQL serverless, SSL)
Orchestration	Docker Compose	Railway (build + déploiement automatique)

Pipeline CI/CD (GitHub Actions)



2.1 Tableau de bord sécurité



8-10 Bon 5-7 À améliorer 0-4 Critique

2.2 Problèmes critiques



CRITIQUE — Secrets committés dans le dépôt Git

Les fichiers `fleet-mgt-api/.env` et `docker-compose.yml` contenant des credentials ont été committés dans l'historique git :

- `DB_PASSWORD` : mot de passe MySQL en clair
- `MAIL_PASSWORD` : clé API Brevo (SMTP)
- `MYSQL_PASSWORD` : exposé dans `docker-compose.yml`

Risque : Toute personne ayant accès au dépôt (présent ou passé via l'historique git) possède ces credentials.

Actions immédiates :

- Révoquer et régénérer TOUS les credentials exposés
- Purger l'historique git avec `git filter-repo`
- Vérifier que `.env` est bien dans `.gitignore`
- Ne conserver qu'un `.env.example` avec des valeurs fictives



MOYEN — Email admin hardcodé dans le frontend

Le fichier `fleet-mgt-frontend/src/components/PrivateRoute.jsx` (ligne 6) contient une adresse email en dur dans le code source.

Remédiation : Passer en variable d'environnement `VITE_ADMIN_EMAIL`.

2.3 Points forts de sécurité

Protection brute-force (OWASP NIST SP 800-63B)

Étape	Comportement
Tentatives 1-4	Message d'erreur générique (sans indication)
Tentative 5 (bloc 1)	Compte bloqué 5 minutes — email envoyé à l'utilisateur
Bloc 2	Blocage 30 minutes
Bloc 3	Blocage 2 heures
Bloc 4+	Blocage 24h puis permanent
Rate limiting	10 req/min sur /login, 5 req/min sur /forgot-password

Politique de mots de passe

- ✓ Minimum 8 caractères
- ✓ Majuscules + minuscules obligatoires
- ✓ Chiffres obligatoires
- ✓ Caractères spéciaux obligatoires
- ✓ Vérification base de données passwords compromis (`uncompromised()`)

Headers de sécurité HTTP

```
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=(), camera=()
Strict-Transport-Security: max-age=31536000; includeSubDomains
Content-Security-Policy: default-src 'self'; script-src 'self'; ...
```

Stockage des tokens

Donnée	Stockage	Durée de vie
user object	localStorage	Jusqu'à logout explicite
access_token	sessionStorage	Fermeture navigateur ou 30 min inactivité
XSRF-TOKEN	Cookie (HttpOnly)	Session
Mots de passe	Bcrypt (DB)	BCRYPT_ROUNDS=12
Sessions Laravel	Table DB chiffrée	SESSION_ENCRYPT=true

2.4 Recommandations

Priorité	Action	Échéance
IMMÉDIAT	Révoquer credentials DB + Brevo, purger historique git	Aujourd'hui
IMMÉDIAT	Passer ADMIN_EMAIL en variable d'environnement	Aujourd'hui
COURT TERME	Restreindre CORS à l'URL de production exacte	Cette semaine
COURT TERME	Ajouter rate limiting sur les routes non-auth	Cette semaine
MOYEN TERME	Implémenter Content-Security-Policy stricte (nonces)	Ce mois
MOYEN TERME	Centraliser les logs (Sentry ou SIEM)	Ce mois
LONG TERME	Tests de pénétration formels	Prochaine release

3.1 Performance

Frontend

Indicateur	État actuel	Opportunité d'amélioration
Build Vite (ES modules)	Optimal	—
Code splitting par routes	Non configuré	<code>React.lazy()</code> sur chaque page
Cache assets statiques	1 an (JS/CSS/images)	—
Tree-shaking Bootstrap	Import global	Importer uniquement les composants utilisés
Lazy loading images	Non évalué	Attribut <code>loading="lazy"</code>
Analyse bundle size	Non mesuré	<code>vite-bundle-visualizer</code>

Backend

Indicateur	État actuel	Recommandation
Eager loading (N+1)	Utilisé avec <code>with()</code>	—
Pagination	15 items/page	—
Cache Laravel	<code>CACHE_STORE=database</code>	Migrer vers Redis
OPCache PHP	Activé en production	—
Index base de données	Non vérifié	Ajouter sur <code>vehicle_id</code> , <code>user_id</code> , <code>date</code>
Exports PDF/Excel	Synchrones (bloquants)	Laravel Queue + Redis

Quick win — Lazy loading des routes

```
// Avant (import statique – charge tout au démarrage)
import Dashboard from './pages/Dashboard';

// Après (import dynamique – charge à la demande)
const Dashboard = React.lazy(() => import('./pages/Dashboard'));

// Dans App.jsx, envelopper les routes :
<Suspense fallback=<Loader />>
  <Route path="/dashboard" element=<Dashboard /> />
</Suspense>
```

3.2 Modernité de la stack

8/10

Stack très récente — peu de dette technique

Technologie	Version utilisée	Dernière stable	État
React	19.1.0	19.x	À jour
React Router	7.6.2	7.x	À jour
Vite	6.3.5	6.x	À jour
Laravel	12.0	12.x	À jour
PHP	8.2	8.4	Migrer vers 8.3/8.4
Bootstrap	5.3.8	5.3.x	À jour
Node	20	22 LTS	Migrer vers Node 22

Points positifs

- ✓ React 19 avec le nouveau compilateur (performances natives améliorées)
- ✓ Laravel 12 — toutes les dernières fonctionnalités de sécurité
- ✓ Vite 6 — build ultra-rapide, HMR instantané
- ✓ Internationalisation EN/FR intégrée (rare à cette échelle)
- ✓ Aucune dette technique majeure visible
- ✓ Déploiement cloud géré : Railway (API), Cloudflare Pages (front), Neon (DB)
- ✓ Suite de tests complète : backend PHPUnit (81 tests) + frontend Vitest (26 tests)

Points d'amélioration

- ✗ Pas de documentation API (OpenAPI/Swagger absent)
- ✗ Pas de TypeScript (migration progressive recommandée)
- ✗ PHP 8.2 — migrer vers 8.3/8.4 pour les performances JIT

3.3 Scalabilité

Capacité estimée

Métrique	Architecture actuelle	Avec Redis + Queue
Utilisateurs simultanés	~500	~5 000+
Requêtes par seconde	~200	~1 000+
Taille de flotte	Illimitée (pagination)	Illimitée
Génération de rapports	Synchrone (bloquant)	Asynchrone (non-bloquant)
Haute disponibilité DB	Neon (serverless auto-scale)	Neon + replica lecture

Points de contention identifiés

- 1

Sessions en base de données
SESSION_DRIVER=database crée une écriture en DB à chaque requête. Goulot d'étranglement au-delà de ~500 utilisateurs simultanés. **Solution** : SESSION_DRIVER=redis
- 2

Exports PDF/Excel synchrones
La génération bloque le worker PHP pendant plusieurs secondes. **Solution** : Laravel Queue + Redis (traitement asynchrone, polling frontend)
- 3

SPA sans code splitting
Le bundle JavaScript charge la totalité de l'application au premier accès. Augmente le Time to Interactive. **Solution** : React.lazy() + Suspense par route

Feuille de route scalabilité

Phase	Actions	Impact
Phase 1 Immédiat	Passer CACHE_STORE et SESSION_DRIVER en Redis Configurer Laravel Horizon pour les queues	×10 capacité
Phase 2 Court terme	Code splitting React (lazy loading routes) Index DB sur colonnes filtrées	TTI –40%
Phase 3 Moyen terme	API rate limiting global Monitoring APM (Sentry, Telescope) Documentation OpenAPI auto-générée	Observabilité
Phase 4 Long terme	CDN pour les exports (R2/S3) Microservices si flotte > 10k véhicules	Enterprise scale

3.4 Synthèse globale



 **Action prioritaire absolue**

Révoquer immédiatement les credentials exposés dans le `.env` commité et purger l'historique git. Cette action ne peut pas attendre.

4.1 Tests Backend — PHPUnit

Le backend est testé avec **PHPUnit 11** via `php artisan test`. Chaque test utilise `RefreshDatabase` sur une base SQLite in-memory (`.env.testing`) pour garantir l'isolation.

Feature/AuthTest — 16 tests

Groupe	Test	Assertion clé
Register	Création utilisateur + token	201 + assertJsonStructure
	Email dupliqué	422 + erreur email
	Mot de passe faible	422 + erreur password
	Rôle par défaut = driver	assertDatabaseHas role=driver
	Rôle admin interdit à l'inscription	422 + erreur role
Login	Connexion valide → token	200 + access_token
	Mauvais mot de passe	401 + INVALID_CREDENTIALS
	Email inconnu	401 + INVALID_CREDENTIALS
	Incrémente login_attempts	assertEquals(1, attempts)
	Réinitialise attempts au succès	assertEquals(0, attempts)
Brute-force	Blocage après 5 tentatives	423 + ACCOUNT_BLOCKED + notification envoyée
	Compte bloqué refuse même le bon MDP	423 + ACCOUNT_BLOCKED
	Accès restauré après expiration du blocage	200 (blocked_until dans le passé)
Logout	Révocation du token Sanctum	assertDatabaseMissing(personal_access_tokens)
/me	Retourne l'utilisateur authentifié	200 + user.id + user.email
	Non authentifié	401

Feature/VehicleControllerTest — 26 tests

Groupe	Test	Résultat attendu
Index	Admin liste les véhicules	200 + {data, total}
	Manager liste les véhicules	200
	Accountant liste les véhicules	200
	Driver bloqué	403
	Non authentifié	401
Store	Admin crée un véhicule	201 + license_plate
	Manager crée un véhicule	201
	Accountant bloqué	403

Groupe	Test	Résultat attendu
	Driver bloqué	403
	Plaque dupliquée	422
	Type de carburant invalide	422
	Kilométrage négatif	422
Show	Tout utilisateur authentifié peut consulter	200 + license_plate
Update	Admin met à jour	200 + mileage=99999
	Manager met à jour le statut	200
	Driver bloqué	403
Destroy	Admin supprime	200 + assertDatabaseMissing
	Manager bloqué	403
Assign Driver	Admin assigne un chauffeur	200 + current_driver_id
	Non-driver refusé	422
	Chauffeur déjà assigné ailleurs	422
	Driver ne peut pas assigner	403
Update Mileage	Chauffeur assigné met à jour	200 + new_mileage
	Kilométrage ne peut pas baisser	422
	Chauffeur non assigné bloqué	403
Expiring Docs	Retourne véhicules avec documents expirant sous 30 jours	count=1 (60 j ignoré)

Feature/UserControllerTest — 21 tests

Groupe	Tests couverts
Index	Admin liste · Manager/Driver bloqués · Non authentifié · Recherche par nom
Store	Admin crée · Manager bloqué · Email dupliqué · Rôle invalide
Show	Admin voit tous · Utilisateur voit son profil · Profil tiers interdit (403)
Update	Mise à jour du nom · Driver ne peut pas changer son rôle · Admin change le rôle
Destroy	Soft delete admin (assertSoftDeleted) · Manager bloqué (403)
Change Password	Succès · Mauvais MDP actuel (401) · Même MDP (422)
Drivers List	Retourne uniquement les chauffeurs (count=3 sur 5 users)

Unit/UserTest — 15 tests & Unit/VehicleTest — 3 tests

Classe	Méthodes testées	Cas couverts
UserTest	isAdmin, isManager, isDriver, isAccountant, isMechanic, hasRole	Vrai pour rôle exact · faux pour tous les autres · hasRole string/array/tableau vide

Classe	Méthodes testées	Cas couverts
VehicleTest	isOperational	operational=true · maintenance=false · out_of_service=false

4.2 Tests Frontend — Vitest + Testing Library

Le frontend est testé avec **Vitest** et **@testing-library/react**. Les timers JS sont mockés via `vi.useFakeTimers()`. Les requêtes Axios sont stubées avec `vi.mock()`.

PrivateRoute.test.jsx — 7 tests

Scénario	Vérification
Utilisateur non connecté	Redirection /login · contenu protégé absent du DOM
Utilisateur connecté sans restriction de rôle	Contenu rendu
Rôle autorisé (admin → route admin)	Contenu rendu
Rôle non autorisé (driver → route admin)	AccessDenied visible · contenu absent
Manager accède route admin+manager	Contenu rendu
Driver bloqué route admin+manager	Contenu absent
Tous rôles acceptés sans prop <code>roles</code>	Contenu rendu pour admin, manager, driver

AuthContext.test.jsx — 6 tests

Scénario	Vérification
Pas d'utilisateur si /me échoue	<code>user = "none"</code>
Restauration utilisateur via /me au démarrage	<code>user.email = "a@a.com"</code>
<code>login()</code> stocke token dans <code>sessionStorage</code>	<code>sessionStorage.getItem("token") = "tok123"</code>
<code>login()</code> stocke user dans <code>localStorage</code>	<code>JSON.parse(localStorage).email</code> correct
<code>logout()</code> efface user et token	<code>user = "none"</code> · <code>sessionStorage</code> vide
<code>logout()</code> fonctionne même si <code>/api/logout</code> échoue réseau	Déconnexion locale garantie

useInactivityTimer.test.js — 6 tests

Scénario	Vérification
<code>onWarn</code> après 28 min d'inactivité	<code>onWarn</code> appelé 1× · <code>onLogout</code> non déclenché
<code>onLogout</code> après 30 min d'inactivité	<code>onLogout</code> appelé 1×
<code>mousemove</code> remet le timer à zéro	27 min + <code>mousemove</code> + 27 min → aucun callback
<code>enabled=false</code> ne démarre pas les timers	Aucun callback après 30 min
Nettoyage des timers au démontage	<code>unmount</code> → aucun callback après 30 min
<code>enabled false</code> → <code>true</code> relance les timers	<code>onLogout</code> déclenché après <code>rerender enabled=true</code>

SessionWarningModal.test.jsx — 7 tests

Scénario	Vérification
visible=false → modal absente	queryByText null
visible=true → modal affichée	"Session sur le point d'expirer" visible
Deux boutons d'action présents	"Rester connecté" + "Se déconnecter"
Clic "Rester connecté" → onStay appelé	vi.fn appelé 1×
Clic "Se déconnecter" → onLogout appelé	vi.fn appelé 1×
Compte à rebours affiché	Format mm:ss ou Xs présent dans le DOM
Réinitialisation compte à rebours (visible false → true)	"2:00" affiché après réouverture

4.3 Synthèse de la couverture



Répartition par couche testée

Couche	Framework	Fichier	Tests	Couverture
Auth API	PHPUnit Feature	AuthTest.php	16	Complète
Véhicules API	PHPUnit Feature	VehicleControllerTest.php	26	Complète
Utilisateurs API	PHPUnit Feature	UserControllerTest.php	21	Complète
Modèles métier	PHPUnit Unit	UserTest.php · VehicleTest.php	18	Complète
Routage protégé	Vitest + RTL	PrivateRoute.test.jsx	7	Complète
Contexte Auth	Vitest + RTL	AuthContext.test.jsx	6	Complète
Hook inactivité	Vitest	useInactivityTimer.test.js	6	Complète
Modal session	Vitest + RTL	SessionWarningModal.test.jsx	7	Complète

Zones non couvertes (à adresser)

- ✗ Maintenances et Consommations : pas de tests Feature dédiés
- ✗ Rapports : pas de tests pour la génération PDF/Excel
- ✗ Pages React (Home, Dashboard, UserDetail...) : pas de tests d'intégration
- ✗ Tests E2E (Playwright/Cypress) absents — parcours utilisateur complet non couvert

!

Recommandation — CI des tests

Ajouter un job GitHub Actions dédié : `php artisan test --coverage` (backend) et `vitest run` (frontend) à chaque Pull Request pour bloquer les régressions avant merge.